

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! ↗



Type '/' to search projects



Help

Docs

Sponsors

Log in

Register

pydensecrf2 1.1



Latest version

`pip install pydensecrf2`

Released: Dec 23, 2024

A python interface to Philipp Krähenbühl's fully-connected (dense) CRF code.

Navigation

≡ Project description

🕒 Release history

⬇️ Download files

Verified details

These details have been
[verified by PyPI](#)

Maintainers

Project description

PyDenseCRF2

NOTE: this code is forked from <https://github.com/lucasb-eyer/pydensecrf>, which is not actively maintained. However, this is a highly useful dependency for our work, [CellSeg3D](#), thus we are maintaining the package for `pypi` here!

Quick start:

```
pip install pydensecrf2
import pydensecrf
```

Original Readme:

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! ↗

Unverified details

These details have **not** been verified by PyPI

Project links

 [Homepage](#)

Meta

- **License:** MIT License
- **Author:** [Lucas Beyer, Philipp Krähenbühl](#) 
- **Maintainer:** [Mackenzie Mathis](#) 

Classifiers

Development Status

- [5 - Production/Stable](#)

Intended Audience

- [Developers](#)

License

- [OSI Approved :: MIT License](#)

Operating System

- [POSIX :: Linux](#)

Programming Language

- [C++](#)
- [Python](#)

Topic

If you use this code for your research, please cite:

Efficient Inference in Fully Connected CRFs with Gaussian
Philipp Krähenbühl and Vladlen Koltun
NIPS 2011

and provide a link to this repository as a footnote or a citation.

Installation

The package is on PyPI, so simply run `pip install pydensecrf` to install it.

If you want the newest and freshest version, you can install it by executing:

```
pip install git+https://github.com/lucasb-eyer/pydensecrf
```

and ignoring all the warnings coming from Eigen.

Note that you need a relatively recent version of Cython (at least version 0.22) for this wrapper, the one shipped with Ubuntu 14.04 is too old. (Thanks to Scott Wehrwein for pointing this out.) I suggest you use a [virtual environment](#) and install the newest version of Cython there (`pip install cython`), but you may update the system version by

```
sudo apt-get remove cython
sudo pip install -U cython
```

Problems on Windows/VS

Since this library needs to compile C++ code, installation can be a little more problematic than pure Python packages. Make sure to [have](#)

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

[REGISTER FOR PYCON US! ↗](#)
[ing :: Image](#)
[Recognition](#)

- [Software](#)

[Development ::](#)
[Libraries](#)


Red Hat

Red Hat is a Maintaining sponsor of the Python Software Foundation.

PSF Sponsor · Served ethically

Report project as
malware

Problems on Colab/Kaggle Kernel

`pydensecrf` does not come pre-installed in Colab or Kaggle Kernel. Running `pip install pydensecrf` will result into build failures. Follow these steps instead for Colab/Kaggle Kernel:

```
pip install -U cython
pip install git+https://github.com/lucasb-eyer/pydensecrf
```

Usage

For images, the easiest way to use this library is using the `DenseCRF2D` class:

```
import numpy as np
import pydensecrf.densecrf as dcrf

d = dcrf.DenseCRF2D(640, 480, 5) # width, height, nlabel
```

Unary potential

You can then set a fixed unary potential in the following way:

```
U = np.array(...) # Get the unary in some way.
print(U.shape)    # -> (5, 480, 640)
print(U.dtype)    # -> dtype('float32')
U = U.reshape((5,-1)) # Needs to be flat.
d.setUnaryEnergy(U)

# Or alternatively: d.setUnary(ConstUnary(U))
```

Remember that `U` should be negative log-probabilities, so if you're using probabilities `py`, don't forget to `U = -np.log(py)` them.

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! ↗

Note that the `nlabels` dimension is the first here before the reshape; you may need to move it there before reshaping if that's not already the case, like so:

```
print(U.shape) # -> (480, 640, 5)
U = U.transpose(2, 0, 1).reshape((5,-1))
```

Getting a Unary

There's two common ways of getting unary potentials:

1. From a hard labeling generated by a human or some other processing. This case is covered by `from pydensecrf.utils import unary_from_labels`.
2. From a probability distribution computed by, e.g. the softmax output of a deep network. For this, see `from pydensecrf.utils import unary_from_softmax`.

For usage of both of these, please refer to their docstrings or have a look at [the example](#).

Pairwise potentials

The two-dimensional case has two utility methods for adding the most-common pairwise potentials:

```
# This adds the color-independent term, features are the
d.addPairwiseGaussian(sxy=(3,3), compat=3, kernel=dcrf.D

# This adds the color-dependent term, i.e. features are
# im is an image-array, e.g. im.dtype == np.uint8 and im
d.addPairwiseBilateral(sxy=(80,80), srgb=(13,13,13), rgb-
```

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! ↗

```
d.addPairwiseGaussian(sxy=3, compat=3)
d.addPairwiseBilateral(sxy=80, srgb=13, rgbim=im, compat=
```

The parameters map to those in the paper as follows: `sxy` in the `Gaussian` case is θ_{γ} , and in the `Bilateral` case, `sxy` and `srgb` map to θ_{α} and θ_{β} , respectively. The names are shorthand for "x/y standard-deviation" and "rgb standard-deviation" and for reference, the formula is:

$$k(\mathbf{f}_i, \mathbf{f}_j) = \underbrace{w^{(1)} \exp \left(-\frac{|p_i - p_j|^2}{2\theta_{\alpha}^2} - \frac{|I_i - I_j|^2}{2\theta_{\beta}^2} \right)}_{\text{appearance kernel}} + \underbrace{w^{(2)} \exp \left(-\frac{|p_i - p_j|^2}{2\theta_{\gamma}^2} \right)}_{\text{smoothness kernel}}.$$

Non-RGB bilateral

An important caveat is that `addPairwiseBilateral` only works for RGB images, i.e. three channels. If your data is of different type than this simple but common case, you'll need to compute your own pairwise energy using `utils.create_pairwise_bilateral`; see the [generic non-2D case](#) for details.

A good [example of working with Non-RGB data](#) is provided as a notebook in the examples folder.

Compatibilities

The `compat` argument can be any of the following:

- A number, then a `PottsCompatibility` is being used.
- A 1D array, then a `DiagonalCompatibility` is being used.
- A 2D array, then a `MatrixCompatibility` is being used.

These are label-compatibilites $\mu(x_i, x_j)$ whose parameters could possibly be [learned](#). For example, they could indicate that mistaking `bird` pixels for `sky` is not as bad as mistaking `cat` for `sky`. The arrays

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! [↗](#)

Kernels

Possible values for the `kernel` argument are:

- `CONST_KERNEL`
- `DIAG_KERNEL` (the default)
- `FULL_KERNEL`

This specifies the kernel's precision-matrix $\Lambda(\mathbf{m})$, which could possibly be learned. These indicate correlations between feature types, the default implying no correlation. Again, this could possibly be [learned](#).

Normalizations

Possible values for the `normalization` argument are:

- `NO_NORMALIZATION`
- `NORMALIZE_BEFORE`
- `NORMALIZE_AFTER`
- `NORMALIZE_SYMMETRIC` (the default)

Kernel weight

I have so far not found a way to set the kernel weights $\mathbf{w}(\mathbf{m})$. According to the paper, $\mathbf{w}(2)$ was set to 1 and $\mathbf{w}(1)$ was cross-validated, but never specified. Looking through Philip's code (included in [pydensecrf/densecrf](#)), I couldn't find such explicit weights, and my guess is they are thus hard-coded to 1. If anyone knows otherwise, please open an issue or, better yet, a pull-request. Update: user @waldol1 has an idea in [this issue](#). Feel free to try it out!

Inference

The easiest way to do inference with 5 iterations is to simply call:

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! [↗](#)

```
map = np.argmax(Q, axis=0).reshape((640,480))
```

If you're interested in the class-probabilities `Q`, you'll notice `Q` is a wrapped Eigen matrix. The Eigen wrappers of this project implement the buffer interface and can be simply cast to numpy arrays like so:

```
proba = np.array(Q)
```

Step-by-step inference

If for some reason you want to run the inference loop manually, you can do so:

```
Q, tmp1, tmp2 = d.startInference()
for i in range(5):
    print("KL-divergence at {}: {}".format(i, d.klDivergence(Q, tmp1, tmp2)))
    d.stepInference(Q, tmp1, tmp2)
```

Generic non-2D

The `DenseCRF` class can be used for generic (non-2D) dense CRFs. Its usage is exactly the same as above, except that the 2D-specific pairwise potentials `addPairwiseGaussian` and `addPairwiseBilateral` are missing.

Instead, you need to use the generic `addPairwiseEnergy` method like this:

```
d = dcrf.DenseCRF(100, 5) # npoints, nlabels
```

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! ↗

```
dcrt.addPairwiseEnergy(feats)
```

In addition, you can pass `compatibility`, `kernel` and `normalization` arguments just like in the 2D gaussian and bilateral cases.

The potential will be computed as $w \cdot \exp(-0.5 * |f_i - f_j|^2)$.

Pairwise potentials for N-D

User @markusnagel has written a couple numpy-functions generalizing the two classic 2-D image pairwise potentials (gaussian and bilateral) to an arbitrary number of dimensions: `create_pairwise_gaussian` and `create_pairwise_bilateral`. You can access them as `from pydensecrf.utils import create_pairwise_gaussian` and then have a look at their docstring to see how to use them.

Learning

The learning has not been fully wrapped. If you need it, get in touch or better yet, wrap it and submit a pull-request!

Here's a pointer for starters: issue#24. We need to wrap the gradients and getting/setting parameters. But then, we also need to do something with these, most likely call [minimizeLBFGS from optimization.cpp](#). It should be relatively straightforward to just follow the learning examples included in the [original code](#).

Common Problems

`undefined symbol` when importing

If while importing `pydensecrf` you get an error about some undefined symbols (for example `.../pydensecrf/densecrf.so: undefined symbol: _ZTINSt8ios_base7failureB5cxx11E`), you most likely are

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

REGISTER FOR PYCON US! ↗

ValueError: Buffer dtype mismatch, expected 'float' but got 'double'

This is a pretty [common](#) user error. It means exactly what it says: you are passing a `double` but it wants a `float`. Solve it by, for example, calling `d.setUnaryEnergy(U.astype(np.float32))` instead of just `d.setUnaryEnergy(U)`, or using `float32` in your code in the first place.

My results are all pixelated like [MS Paint's airbrush tool](#)!

You screwed up reshaping somewhere and treated the class/label dimension as spatial dimension. This is you misunderstanding NumPy's memory layout and nothing that PyDenseCRF can detect or help with.

This mistake often happens for the Unary, see the [Note in that section of the README](#).

Maintaining

These are instructions for maintainers about how to release new versions. (Mainly instructions for myself.)

```
# Go increment the version in setup.py
> python setup.py build_ext
> python setup.py sdist
> twine upload dist/pydensecrf-VERSION_NUM.tar.gz
```

And that's it. At some point, it would be cool to automate this on [TravisCI](#), but not worth it yet. At that point, looking into [creating "manylinux" wheels](#) might be nice, too.

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they're gone!

[REGISTER FOR PYCON US! ↗](#)

package and run `py.test`. Maybe there's a better way to run them, but both of us don't know :smile:



Help

[Installing packages ↗](#)
[Uploading packages ↗](#)
[User guide ↗](#)
[Project name retention ↗](#)
[FAQs](#)

About PyPI

[PyPI Blog ↗](#)
[Infrastructure dashboard ↗](#)
[Statistics](#)
[Logos & trademarks](#)
[Our sponsors](#)

Contributing to PyPI

[Bugs and feedback](#)
[Contribute on GitHub ↗](#)
[Translate PyPI ↗](#)
[Sponsor PyPI](#)
[Development credits ↗](#)

Using PyPI

[Terms of Service ↗](#)
[Report security issue](#)
[Code of conduct ↗](#)
[Privacy Notice ↗](#)
[Acceptable Use Policy ↗](#)

Status: All Systems Operational [↗](#)

Developed and maintained by the Python community, for the Python community.
[Donate today!](#)

"PyPI", "Python Package Index", and the blocks logos are registered trademarks of the Python Software Foundation [↗](#).

📌 Join us in Long Beach, CA starting May 13, 2026. Grab your ticket and discounted hotel today before they’re gone!

REGISTER FOR PYCON US! ↗

Switch to desktop version

> English
 español
 français
 (
)
 português (Brasil)
 українська
 Ελληνικά
 Deutsch
 (
)
 русский
 עברית
 Esperanto
 한국어

AWS
Cloud computing
and Security
Sponsor

Datadog
Monitoring

Depot
Continuous
Integration

Fastly
CDN

Google
Download Analytics

Pingdom
Monitoring

Sentry
Error logging

StatusPage
Status page